training-agent-mcp

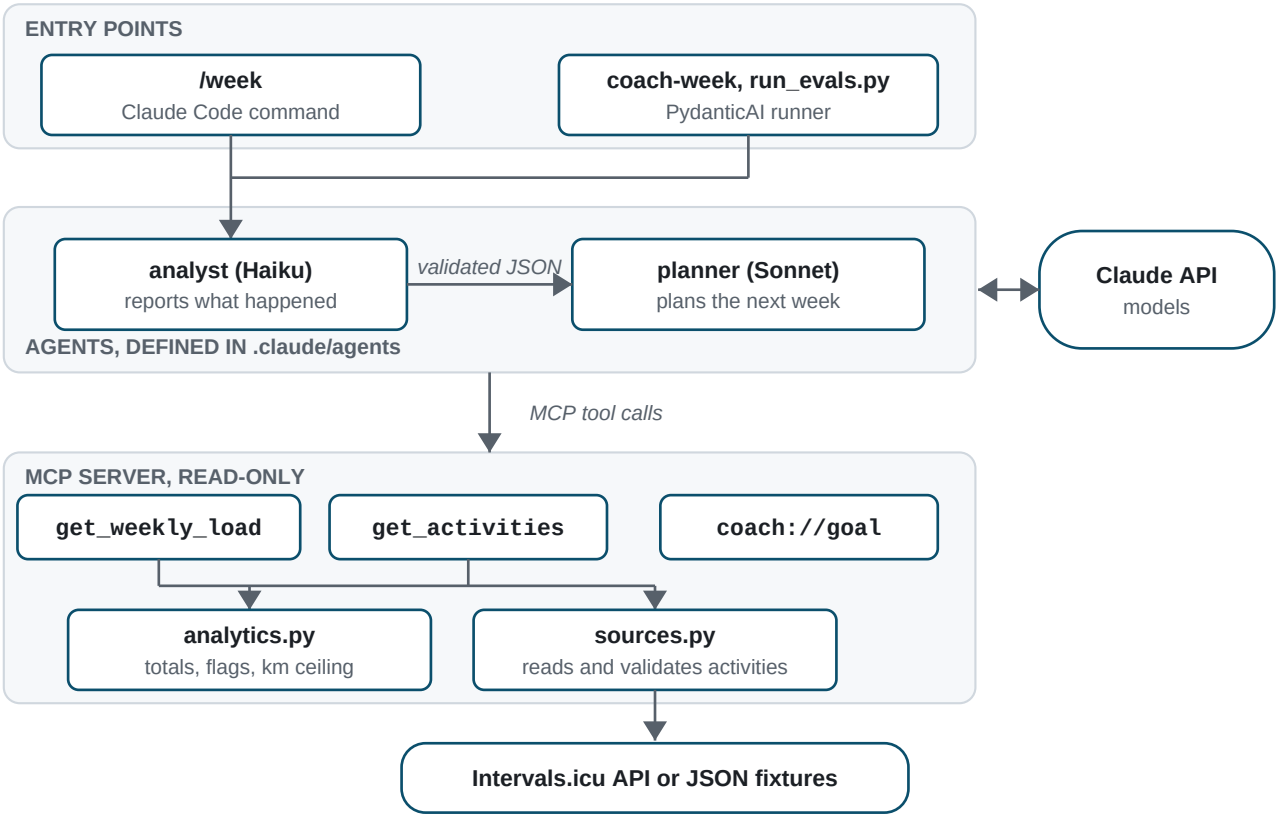
A read-only MCP server over my running data, with two agents that review a training week and plan the next one.

The problem

I worked on an enterprise MCP server with over a hundred tools and wanted to try the opposite: a small server with a couple of tools. This project answers two questions from my training data: how was my week, and what should next week look like?

I set the requirements, made the design calls and ran the evals. Claude Code generated most of the code and wrote the eval cases.

Architecture



Two entry points run the same agents. Both agents reach the data only through the server, and every number they see is computed there.

How one run works

Given an ISO week, the analyst calls the two tools and returns a JSON object with a fixed set of fields. The runner validates it with Pydantic and retries once if it does not match. The planner receives that JSON, reads the target race and returns seven days plus its reasoning, also validated. A run on the synthetic data takes about 13 seconds.

Code does the arithmetic

Weekly totals, the percentage change, the zero-week streak, the flags and the ceiling for next week's distance are pure functions in `analytics.py`. The models copy these values and interpret them. The one flag a model decides, `race_detected`, travels in a separate field so it is always marked as an inference.

Situation	What the server returns	What the planner must do
Normal week	Ceiling: the larger of the recent average and this week, plus 10 %	Stay at or below the ceiling
Load spike or layoff	Ceiling: the recent average, with no increase	Plan recovery, or ask why the runner stopped
Unreadable activities	No ceiling, and a fixed 10 km safety cap	Say the load is unknown and plan easy runs under the cap

Design decisions

Two tools

I decided this up front, as a reaction to the large server. `get_weekly_load` answers most questions and `get_activities` inspects specific runs. The race goal is static data, so it is an MCP resource. I did not measure this against a larger tool set, so it is a design choice and not a benchmark result.

Two agents

Separate agents keep raw tool output out of the planner's context and let extraction run on a cheaper model. In a typical run the analyst costs about $0.001 and the planner about $0.02.

One definition per agent

The Claude Code command and the Python runner read the same files for prompt, model and tool allowlist, so they cannot drift apart. The runner replaces only the planner's output format, because it needs validated fields.

Read-only, with least privilege

The server only issues GET requests. Each agent gets an explicit tool allowlist. In Claude Code, a hook also limits how much activity data one call can pull into context.

Errors say how to fix the call

A range that is too long, a bad week format, a rate limit or a missing goal file each return a message the agent can act on. A rate limit passes on the wait from the server, and only if it is a number, because that header is remote text that reaches the model.

Missing data stays explicit

An activity that exists but cannot be read is reported as blocked. It never counts as a rest day, it stops the percentage change and the ceiling from being computed, and it breaks the zero-week streak. Heart rate stays null when there is no sensor and is never estimated.

Stack

Python, Pydantic, the MCP Python SDK, PydanticAI and Logfire are the tools I work with. I left out LangChain because native tool calling and structured output already cover what this project needs. The code passes `mypy --strict` and has 140 offline tests.

Evals

Each case runs the real agents and the real MCP server on its own data, and scores the validated output. A case holds the data, the question, the expected behaviour and a list of criteria, all under a Pydantic schema. A dry run compares expected values with what the server computes, without calling a model.

Check	When to use it	Example
Code	Whenever the answer can be written as a comparison. It is free and repeatable, and wording cannot fool it.	Flags are exactly `["layoff"]`. The plan totals 1 to 6 km.
LLM judge	Only for meaning that code cannot decide. It runs after the code checks pass, because it costs money and can be wrong.	The question asks why the runner stopped.

Failures of the plumbing, such as a rate limit or a timeout, count as errors and stay out of the pass rate. Plan totals have a lower limit as well as an upper one, otherwise an empty plan would pass.

Results

Set	Cases	Attempts	Passed	Cost
Synthetic, in this repo	5	15	15	$0.33
My real data, kept out of the repo	6	6	6	$0.15

Measured on 2026-11-08. The cases cover a normal week, a layoff, a week exactly at the spike threshold, a race close by, a race after low volume and several kinds of unreadable data.

What the evals caught

The earlier runs failed three times, for different reasons.

- The planner had to plan below last week's distance, and with unreadable data that distance was 0 km. It said so and invented 9 to 13 km. The fix was in code: the server now returns a 10 km safety cap.
- One judge rubric rejected a 0.0 km field that the output contract requires, although the agent had described the data correctly. The rubric was rewritten after that failure, which is why the change is noted here.
- A Haiku judge failed a correct answer that a person would pass. This was measured on saved answers, so the agents did not vary.

Saved answer, 10 trials each	Right verdict	Haiku	Sonnet
Correct answer, original case text	Pass	10/10	10/10
Correct answer, corrected case text	Pass	7/10	10/10
Deliberately bad answer	Fail	10/10	10/10

The judge now uses Sonnet, at about $0.004 per verdict against $0.0002. This used two saved answers, enough to choose a model but not to call Sonnet reliable.

Observability

Logfire records every model request and tool call as one trace, including the steps inside the MCP server process. This is a real run, shortened:

```
week 2026-W30                15.75K tokens in, 885 out, 9.78 s
  agent analyst              10.58K in, 422 out, 4.39 s
    MCP connect              425 ms
    chat claude-haiku-5-5    2998 in,  87 out, 1.45 s
    tool get_weekly_load     29 ms
    chat claude-haiku-5-5    3544 in,  86 out, 958 ms
    tool get_activities      13 ms
    chat claude-haiku-5-5    4039 in, 249 out, 1.50 s
  agent planner              5170 in, 463 out, 5.39 s
    MCP connect              435 ms
    chat claude-sonnet-5-5   2504 in,  89 out, 1.80 s
    tool ReadMcpResourceTool 7 ms
    chat claude-sonnet-5-5   2666 in, 374 out, 3.13 s
```

Most of the time goes to the models, and tools take milliseconds. Starting a server process costs about 430 ms per agent. The run cost $0.016, computed from real token counts and a price table, one request at a time because some prices depend on prompt length.

By default traces carry tokens, latency and cost, and never prompts or responses. A test searches the exported spans for known content. `COACH_LOG_CONTENT=true` writes that content to a local, git-ignored file for debugging and sends nothing out.

Limitations

- Intervals.icu does not expose Strava-synced activities through its API. All 52 activities on my account since August 2025 come back with a date and no distance, so with live data the system can only report that data is missing. The real eval cases use six runs I transcribed by hand and those unreadable activities.
- The eval set is small and the cases share one author. Real cases ran once and synthetic cases three times. The results show the system follows its specification on these cases, not that the plans are good training advice.
- The planner tends to fill its limit. With a 6 km ceiling it plans 6 km. There is no taper rule, so the plan for a race 13 days away ranged from 32 to 40 km between runs, and all of those attempts passed.
- The 10 km cap is a cautious project choice, not a validated training rule.
- Cost figures are estimates. They come from token counts and a price table that does not update itself.

Run it

```
uv sync
uv run pytest
ANTHROPIC_API_KEY=your-key uv run python evals/run_evals.py
```

The first two commands need no account. The third costs about $0.11 per pass. Not medical or coaching advice.